

```
In [1]: import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import models, transforms
import matplotlib.pyplot as plt
from PIL import Image
```

读取与展示风格和内容图片

```
In [ ]: content_img = Image.open("content.png").convert("RGB")

style_img = Image.open("style.png").convert("RGB")

# 展示输入图像
plt.figure(figsize=(10,5))
plt.subplot(1,2,1); plt.imshow(content_img); plt.title("Content Image"); plt.axis('off')
plt.subplot(1,2,2); plt.imshow(style_img); plt.title("Style Image"); plt.axis('off')
plt.show()

img_tfm = transforms.Compose([
    transforms.Resize((512, 512)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

content_tensor = img_tfm(content_img).unsqueeze(0).cuda()
style_tensor = img_tfm(style_img).unsqueeze(0).cuda()

# 反归一化
inv_tfm = transforms.Compose([
    transforms.Normalize(mean=[0,0,0], std=[1/0.229,1/0.224,1/0.225]),
    transforms.Normalize(mean=[-0.485,-0.456,-0.406], std=[1,1,1])
])
```

Content Image



Style Image



设置损失网络

```
In [ ]: class LossNet(torch.nn.Module):
    def __init__(self, backbone):
        super(LossNet, self).__init__() # 初始化
        self.select = ['3', '8', '15', '22'] # 选择的层
```

```

self.feature_detector = backbone.features # 提取特征的部分
# 冻结参数
for p in self.parameters():
    p.requires_grad = False

def forward(self, x):
    features = []
    # 前向传播, 提取选定层的特征
    for name, layer in self.feature_detector._modules.items():
        x = layer(x) # 前向传播
        # 如果是选定层, 保存特征
        if name in self.select:
            features.append(x)
    return features

```

风格损失

```

In [ ]: def gram_matrix(x):
    (b, ch, h, w) = x.size() # batch size, channel, height, width
    # 重塑特征图
    features = x.view(b, ch, w * h)
    features_t = features.transpose(1, 2) # 转置
    # 计算Gram矩阵
    gram = features.bmm(features_t) / (ch * h * w)
    return gram

```

训练函数

```

In [ ]: def style_transfer_train(
    model, input_img, style_grams, content_features, optimizer, max_T=1000
):
    style_weight = 1e7 # 风格损失权重
    content_weight = 1 # 内容损失权重

    for t in range(max_T):
        optimizer.zero_grad() # 梯度清零
        # 提取输入图像的特征

        features = model(input_img)
        grams = [gram_matrix(x) for x in features] # 计算Gram矩阵

        # 内容损失
        content_loss = F.mse_loss(features[2], content_features[2]) * content_weight

        # 风格损失
        style_loss = 0 # 初始化风格损失
        # 计算每层的风格损失并累加
        for a, b in zip(grams, style_grams):
            style_loss += F.mse_loss(a, b) * style_weight

        loss = content_loss + style_loss # 总损失

        loss.backward() # 反向传播
        # 更新输入图像
        optimizer.step()

```

```

        if t % 100 == 0:
            print(f"Step {t}, Loss: {loss.item():.4f}") # 每100步打印一次损失

    return input_img

```

```

In [ ]: vgg = models.vgg19(pretrained=True).cuda().eval() # 加载预训练VGG19模型
        lossnet = LossNet(vgg).cuda() # 初始化损失网络

        content = content_tensor.cuda() # 内容图像张量
        style = style_tensor.cuda() # 风格图像张量
        train_input = content_tensor.clone().cuda().requires_grad_(True) # 训练输入图像

        content_features = lossnet(content) # 提取内容特征
        style_features = lossnet(style) # 提取风格特征
        style_grams = [gram_matrix(f) for f in style_features] # 计算风格Gram矩阵

        optimizer = torch.optim.Adam([train_input], lr=0.01) # 优化器

        output = style_transfer_train(
            lossnet, train_input, style_grams, content_features, optimizer, max_T=1000
        ) # 训练

```

```

Step 0, Loss: 601.3288
Step 100, Loss: 53.2126
Step 200, Loss: 27.8820
Step 300, Loss: 18.0493
Step 400, Loss: 13.5191
Step 500, Loss: 11.2820
Step 600, Loss: 10.0266
Step 700, Loss: 9.2871
Step 800, Loss: 8.7204
Step 900, Loss: 8.3524

```

最终的风格迁移结果展示

```

In [ ]: output_img = output.clone().detach().cpu().squeeze(0) # 处理输出图像
        output_img = inv_tfm(output_img) # 反归一化
        output_img = torch.clamp(output_img, 0, 1) # 限制像素值在[0,1]
        output_pil = transforms.ToPILImage()(output_img) # 转换为PIL图像

        plt.figure(figsize=(6,6)) # 显示结果
        plt.imshow(output_pil)
        plt.title("Stylized Result")
        plt.axis("off")
        plt.show()

```

Stylized Result

